Electrophysiology test of Signal Detection Theory. Aim:

In the last practical we measured how well you could detect a weak visual signal (the X). Measuring your threshold was only partially successful because signals are always detected against a background of neuronal noise and because our decisions are influenced by bias. Signal detection theory is a newer description of how signals are represented in the brain and how we make decisions. Using signal detection theory, we measured sensitivity as D'. Hopefully you found that D' did not change so much between the different bias conditions, while the measure of bias 'criterion' was consistently different.

In today's practical we will see if the central idea of signal detection theory can be measured from MEG data. i.e. We will test whether the brain behaves like the theory says it does. You will advance your programming skills, get experience working with electrophysiology data and learn to use the fieldtrip toolbox (http://www.fieldtriptoolbox.org/) which is widely used to analyze all sorts of neuroscience data. We will do analysis in the time domain and in the frequency domain and you will write a script to calculate the receiver operating characteristic (ROC) between two distributions of data.

Signal detection theory is based on the idea that neuronal responses are noisy (variable from one measurement to another) even when there is no stimulus. Responses with no stimulus are therefore described as a noise distribution. With a stimulus present, responses are larger but still noisy, therefore the distribution of responses with a stimulus present (Stimulus + Noise distribution) is shifted to the right, but overlaps with, the noise distribution. Because the two distributions overlap, you can't know for sure if a stimulus is present or not by looking a single response. If the stimulus gets stronger, the Stimulus + Noise distribution shifts further to the right, making it easier to guess whether a stimulus is there or not.

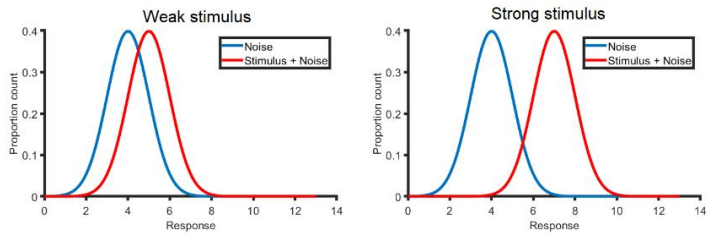


Fig1) Illustration of Signal Detection Theory

To test whether the brain responds like this we need repeated measurements of neuronal activity: 1) Without a stimulus, 2) With a stimulus, 3) With stimuli of different strengths. We will use two measure of neuronal activity; the evoked response and gamma band power in MEG data 1) with no visual stimulus present, 2) with a visual stimulus, 3) with visual stimuli of 5 different contrasts. The experiment is described in Hadjipapas's 2015 neuroimage paper.

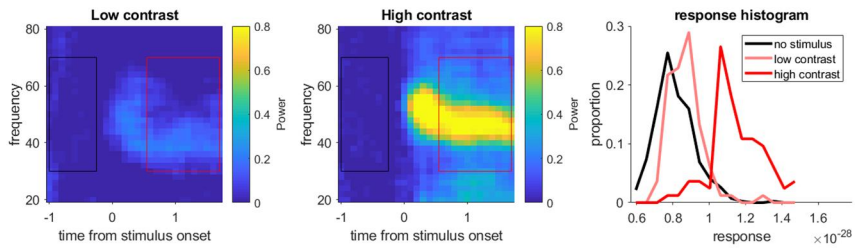


Fig2) MEG response to high and low contrast stimuli. Distributions of responses with no stimulus (i.e. noise) weak stimulus and strong stimulus (i.e. stimulus + noise).

For you report you should include

- one figure that shows the time domain response at each contrast and one figure that shows five responses overlayed.
- One figure that shows the frequency domain response at each contrast and one figure that shows the five responses overlayed.
- One figure that shows the response distributions and ROC curves for a few stimulus strengths. (This is the most important figure, you can use this figure to argue whether the signal detection theory is or is not a good description of how the brain works).

Practice analysis

As in the second session, we start with a practice analysis using fake data. Open your script 'FakeData', saved from the second session. This script makes two distributions of data and compares them with a t-test. Now we will compare them using signal detection theory.

1) Start a new section to the script and make two datasets such that the second set has a higher mean than first set:

```
%%%%%%%%%%%% Part 3 (session 5) %%%%%%%%%%%%%%%%%%%%%%%%
%%   Using two percent signs breaks your script into sections that can be run separatly Ctrl+Enter

clear all, close all, clc
% make some data
data1=randn(1,1000);
data2=randn(1,1000)+2;
```

Suppose you are given a single number and asked to guess whether it comes from data 1 or from data2. If the number is high you might guess it comes from data2, if it is low it might come from data1. You set a *criterion* to decide if the number is 'high' or 'low'. How often will you be correct? If the number is higher than the criterion you guess it comes from data2, if not you guess data1. If the number did come from data2, you score a Hit. Otherwise, you score a False Alarm.

Use the function find to identify values above your criterion. Use the function length to count how many such numbers there are. Divide the number by the length of the data to calculate the rate.

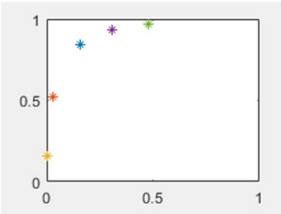
```
Criterion = 1;
Hit_rate= length(find(data2>Criterion))/length(data2)
False_alarm_rate=length(find(data1> Criterion))/length(data1)
```

Plot these numbers as part of a Receiver Operating Characteristic in a new subplot.

```
subplot(2,2,3)
plot(False_alarm_rate, Hit_rate,'*'); hold on
set(gca,'xlim',[0 1],'ylim',[0 1]) % set the limits of the x and y axis
```

Repeat the process with a new criterion. How does this change performance? (you can run a few lines of code by highlighting them and pressing F9)

```
Criterion = 0.1;
Hit_rate= length(find(data2>Criterion))/length(data2)
False_alarm_rate=length(find(data1> Criterion))/length(data1)

subplot(2,2,3)
plot(False_alarm_rate, Hit_rate,'*'); hold on
set(gca,'xlim',[0 1],'ylim',[0 1]) % set the limits of the x and y axis
```



To calculate the full ROC curve we must test every possible criterion. Instead of setting the criteria yourself, use a for loop. Remember that a for loop must end with an end statement. Use criterions from the highest value of data2 to the lowest value of data1.

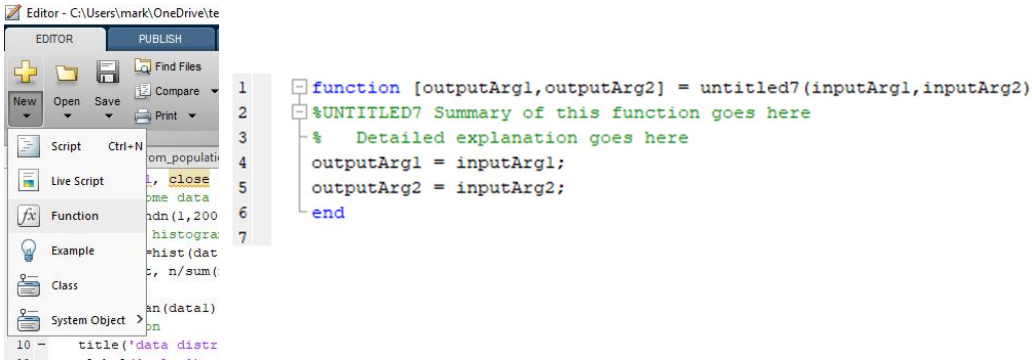
```
for Criterion = sort([data1, data2],'descend') % test every number in the data
    Hit_rate= length(find(data2>Criterion))/length(data2); %add a semicolone to stop writing each result
    False_alarm_rate=length(find(data1> Criterion))/length(data1);

    subplot(2,2,3)
    plot(False_alarm_rate, Hit_rate,'*'); hold on
    set(gca,'xlim',[0 1],'ylim',[0 1]) % set the limits of the x and y axis
end% end the for loop
% you can indent your for loops using 'smart indent' but you don't have to
```

The ROC curve is quantified by the area under the curve (AUC). For this we need to keep the results of each criterion. Start by making an empty array for hits and false alarms. Then you can add a new value to the end of the array for each criterion you test. Use the function trapz to calculate the area under the curve.

```
Hit_rate=[];False_alarm_rate=[];%start with an empty array
for Criterion = sort([data1, data2],'descend') % test every number in the data
    Hit_rate(end+1)= length(find(data2>Criterion))/length(data2); %add a semicolone to stop writing each result
    False_alarm_rate(end+1)=length(find(data1> Criterion))/length(data1);
end% end the for loop
% do the plotting at the end of the for loop
subplot(2,2,3)
plot(False_alarm_rate, Hit_rate,'-'); hold on
set(gca,'xlim',[0 1],'ylim',[0 1]) % set the limits of the x and y axis
AUC=trapz(False_alarm_rate, Hit_rate)
```

The AUC quantifies how well two distributions can be distinguished. Try changing data1 and data2. Try changing the size of each dataset and how far apart they are.

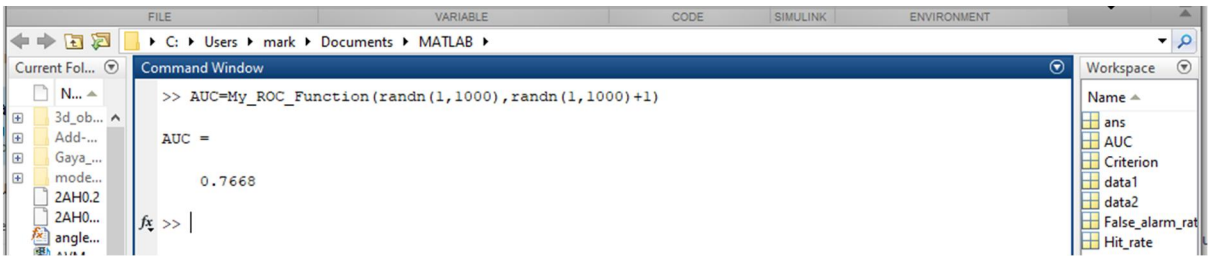
2) Calculating AUC is very useful for all sorts of data analysis, so it is worthwhile having your own function to calculate it. Your function should take two datasets as inputs and return AUC as an output and it should plot both the distributions of the datasets and the ROC curve.
Open a new function using the New button. This gives an outline of how to set up a new function



Replace the default setup with stuff that is relevant to calculating AUC. The output will be AUC and the inputs will be two datasets. Copy and paste the relevant parts from your script.

```
function [AUC] = My_ROC_Function(data1,data2)
%My_ROC_Function calculates AUC for two datasets

% subplot shows the data distributions
subplot(2,2,1)
[n,xout]=hist(data1,25);
plot(xout, n/sum(n))
hold on
[n,xout]=hist(data2,25);
plot(xout, n/sum(n))

% now calculate AUC
subplot(2,2,2)
Hit_rate=[];False_alarm_rate=[];
for Criterion = sort([data1 data2],'descend')
    Hit_rate(end+1)= length(find(data2>Criterion))/length(data2);
    False_alarm_rate(end+1)=length(find(data1> Criterion))/length(data1);
end
plot(False_alarm_rate, Hit_rate,'-'); hold on
AUC = trapz(False_alarm_rate, Hit_rate);

end
```

When you save the function, it is automatically named according to what you put on the first line, in my case 'My_ROC_Function'. You can now use your new function at the command window or from another script.



MEG analysis

The data we will use comes from an experiment in which a participant was shown a luminance pattern of 5 contrasts. Each contrast level was shown many times (trials). Electrophysiology data was recorded with an MEG scanner. The scanner has many sensors (channels) each positioned over a different part of the head and sensitive to a different part of the brain. The data has already been cleaned of artifacts, cut into trials and divided into separate conditions.

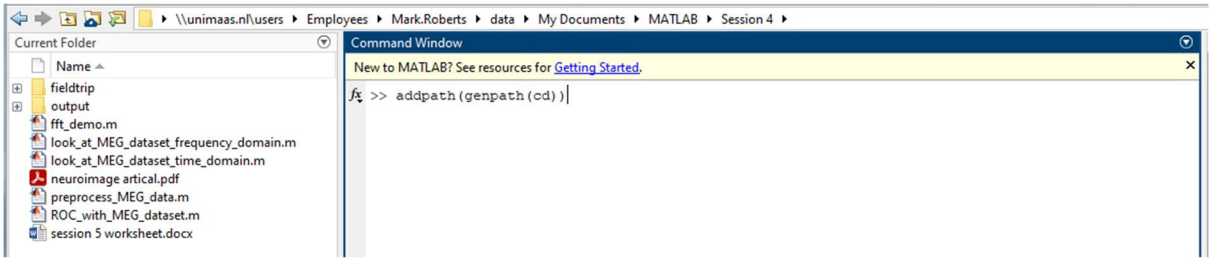
We will use the Fieldtrip toolbox. This suite of MATLAB functions has been developed to make common neuroscience data analysis easy. The steps to the analysis are:

1) Select which channels are most responsive to the stimulus
2) Inspect how these channels respond to stimuli of different contrasts
   a. In the time domain
   b. In the frequency domain
3) Inspect the distribution of responses with and without stimulus, see if this matches predictions from signal detection theory.

Get everything set up.

If you havn't already done so unzip the folder from SurfDRIVE, there are two folders inside the folder of today's session. One has (a much reduced version of) the Fieldtrip toolbox. One has the data we will work with. There are also some prepared scripts.

In the fieldtrip folder there are many more folders and functions. MATLAB needs to know to access these folders. Add all the subfolders to the matlab path using the functions addpath and genpath



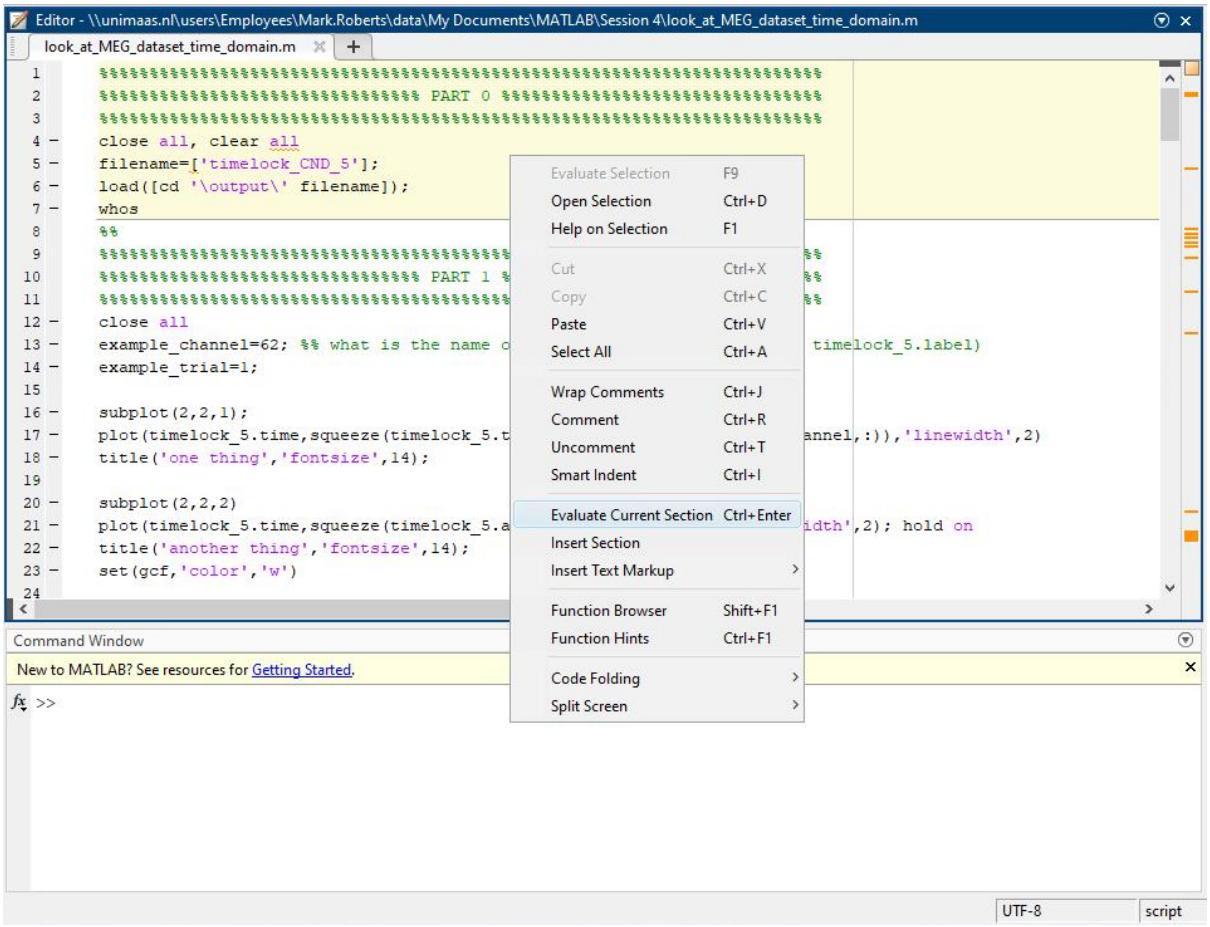
Now MATLAB 'knows' where to find the functions we will use. If you close MATLAB, you will have to use addpath again when you restart MATLAB.

Now start the analysis in the time domain

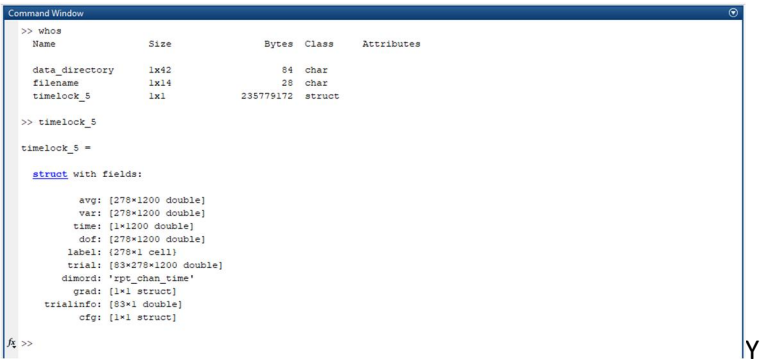
MEG signals represent the combined activity of many neurons, therefore the signal is only really strong when many neurons are active simultaneously. One way that this can happen is that an external stimulus switches on and excites the neurons.

Move back to the main session 4 course that has the prepared scripts

Open the script look_at_MEG_dataset_time_domain.m This script is broken into sections using the %% operator. You can run (evaluate) each section separately by placing the curser in the section (it then is highlighted yellow) and either using the right mouse button to get a drop-down menu or using Ctrl+Enter



The first section of the script to loads some data (this make take a moment) and uses the command whos to see what has been loaded. You have loaded a thing called *timelock_5*. This is a MATLAB structure it has lots of stuff inside it. Type *timelock_5* into the command line to see what it is.



Y

You can see from the structure that there at 278 labels, which correspond to the MEG sensors. And there are 83 trials. On each trial a stimulus was presented. Some parts of the object are cell arrays. You need to use curly brackets to look at a cell array. Other parts are lists of numbers called double. You need to use normal or square brackets for those. Other parts are structures. That means there is another structure inside the first structure.

What is the name of the first channel? In the command line type

```
Command Window
>> timelock_5.label{1}
```

To save memory space the data has been downsampled. What is the original sampling frequency (origfs)? What has it been reduced to (resamplefs)? You have look deeply into the structure to find it.

```
Command Window
>> timelock_5.cfg.previous
```

Part 1

Choose some example channels. Look at a single trial and at the average trial in one condition. What do you notice about how a single trial looks compared to the average? One channel has a strong response after time 0 (when the stimulus switches on), the other does not. We need to select which channels respond.

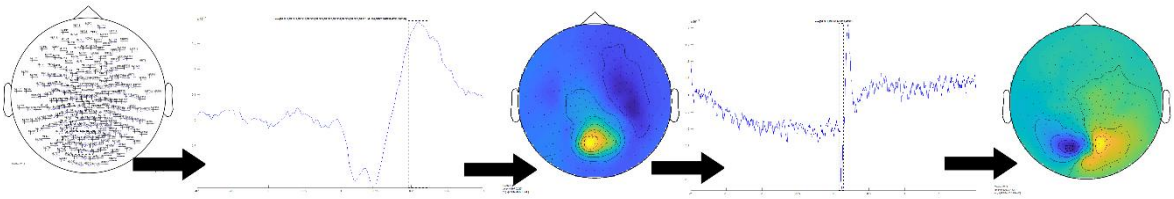
[try to understand this figure, but it is not needed for your report]

Part 2

Now we need to choose the channels that will be most useful for our analysis.

Use the ft_multiplotER function to plot all the channels in their proper place. This is an interactive plot. Highlight a portion of the figure and click on the highlighted region to get an average of the evoked field from the highlighted channels. Highlight a portion of the ERF to get a topoplot of that time period. Highlight a portion of the topoplot to see the ERP of those channels. You can keep doing this until you get a fast-looking ERP and toplot that looks relevant for visual cortex.

Use the first part of the response, i.e. the **negative part**, to select channels and timepoints.



The title of the ERP plot shows the names of the channels that are included and you can copy them from the command window

```
Command Window
the call to "ft_topoplotER" took 1 seconds
selected cfg.channel = {'MRF21'}
the call to "ft_singleplotER" took 1 seconds
selected cfg.channel = {'MRF21', 'MRF22'}
the call to "ft_singleplotER" took 1 seconds
>>
```

(MRF21 and MRF22 are **not** the ones you should choose!)

[These figures are steps in the analysis, you do not need them for your report]

Part 3

Load the data from the other 4 contrast conditions (this might take a moment)

Enter the channels you selected in part 2 and look at how they respond for the five conditions. You can copy the names from the command window and enter them into the script.

```
selected cfg.channel = {'MLF46', 'MLF55', 'MLF56', 'MLF65', 'MLT11', 'MLT12'}
the call to "ft_singleplotER" took 0 seconds
>>
```

```
close all
example_channel_name=[{'MLF46', 'MLF55', 'MLF56', 'MLF65', 'MLT11', 'MLT12'}] %% which channel is intersting to plot? these are not very good channels to choose!
example_channel=find(ismember(timelock_1.label,example_channel_name));
```

Some time-points are more interesting than others. Set the X limits to zoom in on the most interesting parts. Look at how the figure is formatted using the set function. It is possible to do quite some formatting of your figures. These figures are made with a small fontsize. Try setting a larger fontsize by adding ,'fontsize',16 to set. You can see all the things that are available to change by typing get(gca);

That gets all the properties of the current axis (gca stands for get current axis). You can also change the way the figure looks using set(gcf).

CHECK that this figure is automatically saved into your data directory for your report.

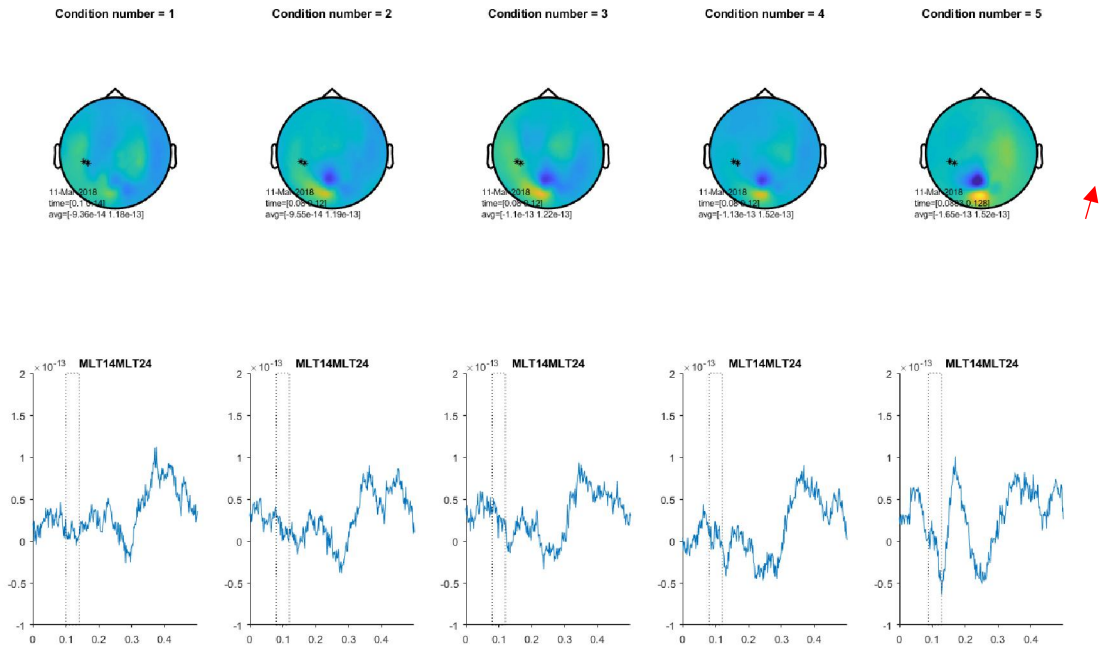
Part 4

To fully understand the response, we need to see its spatial distribution for each contrast using topoplots. However, the time course of the response changes with contrast, so we need to set slightly different time points for each contrast. In part 4 adjust

```
Best_time_windows
```

to identify the time period of the early negative response for each condition. Make sure figures have relevant X and Y labels.

This figure is automatically saved into your data directory for your report. Put your figure into Word (or similar) and use the insert tab to add an arrow along where you think the data suggests a current (the arrow below is not in the right place!).



What does the time-domain analysis tell you about how the brain responds to stimuli of different contrasts? Is the response fundamentally the same or fundamentally different? Is the same area active? How does its response change in strength and latency (the time taken for the response to start)?

CHECK that this figure is automatically saved into your data directory for your report.

Part 5

Now we know 1) which channels are responsive 2) when those channels are responsive. Save that data into the structure output data. Notice that your ROC function expects the stimulus dataset to be larger than the baseline but the neuronal response here is negative. Therefore, the sign of the response has been reversed.

```
output_data.time_CND1=-squeeze(nanmean(nanmean(timelock_1.trial…
```

Use your ROC function to calculate AUC for each contrast condition against the baseline response (you can find the contrast values that were used in the paper).

```
AUC(1)=MY_ROC_Function(output_data.time_CND0,output_data.time_CND1)
AUC(2)=MY_ROC_Function(output_data.time_CND0,output_data.time_CND2)
etc
```

Plot AUC as a function of contrast to show sensitivity of the evoked response to stimulus onsets.

This figure is not *automatically* saved! Annotate it and include it in your report. Does the evoked response behave as signal detection theory predicts?

Going to the frequency domain

A second way in which many neurons can activated simultaneously so that their activity becomes visible in MEG, is when a network of neurons spontaneously synchronizes in a particular rhythm. Normally this rhythm happens at a frequency of between 20 and 70Hz, in which case we refer to it at 'gamma band activity'.

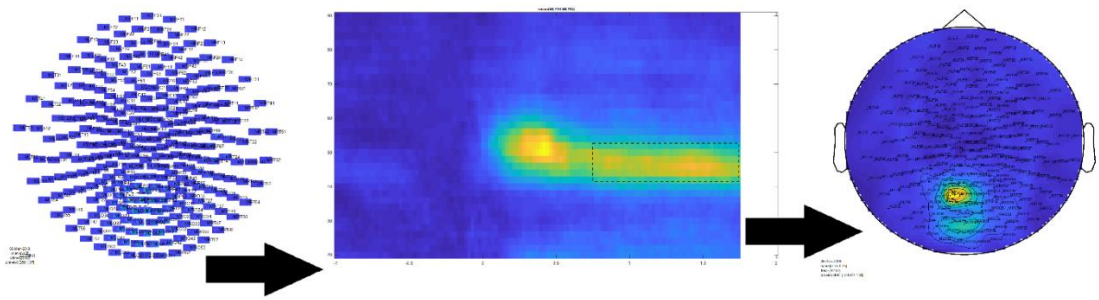
Open the script look_at_MEG_dataset_frequency_doman.m and adjust the data directory

Part 1

Calculating TFRs for all channels takes a while. To save time the data has already been processed in the output folder. If you have the files the script will load them, otherwise it will calculate them.

Part 2

Use the multiplotTFR function to make a multiplot of the time frequency plots from one condition. This is another interactive plot to explore the data with.



Notice that we are more interested in the **late part** of the response – the self-sustained gamma band rhythm, than the early part, which indicates the stimulus-evoked response.

[These figures are steps in the analysis, you do not need them for your report]

Part 3

Chose some channels that seem responsive (based on the multiplotter). Put the names into `example_channel_name`

Are these the same channels you used for the time-domain analysis?

Choose some times and frequencies that seem responsive. Put these into `cfg.xlim` and `cfg.ylim`

Make sure you mention what settings you used in your report.

Run part 3 to produce a plot for each condition and a summary plot showing the baseline-corrected power spectrum for each condition. How does contrast affect gamma activity in terms of active sites, power and frequency?

Annotate these plots. **Check** that they are saved automatically to include in your report.

Part 4

Now we know 1) which channels are responsive 2) when those channels are responsive 3) which frequencies are responsive. Save that data into the structure output data

Use your ROC function to calculate AUC for each contrast condition.
Plot AUC as a function of contrast to show sensitivity of the evoked response to stimulus onsets. This figure is not automatically saved! Annotate it and include it in your report. Does the gamma band response behave as signal detection theory predicts?
You calculated AUC in both the time domain and in the frequency domain. Which is more sensitive? Can you make a figure comparing the them?